

Fetal_Assessment_Project

08/21/2026 22:05:37

<https://chatgpt.com/c/6a4d1688-4488-83e8-aeb2-2606b4d69a23>

Absolutely. The most important thing now is to stop thinking of this as a...



Absolutely. The most important thing now is to **stop thinking of this as a collection of notebooks/scripts** and start thinking of it as a **validated end-to-end fetal movement analysis system** that is being built in stages.

Your final destination is:

Real maternal ACC + GYRO data → reliable signals → artifact separation → fetal movement events → movement characteristics → temporal behavioral profile → evidence-based interpretation → future parent-facing visualization/app

But you have only ~7-8 days before real maternal data may arrive. So the strategy should be to make the software **ready to accept real data**, while using the available datasets to validate each layer.

Fetal Assessment Project — Master Execution Strategy

0. The North Star

Final technical goal

Build a software pipeline capable of receiving:

MPU6050

- Accelerometer: `ax, ay, az`
- Gyroscope: `gx, gy, gz`
- timestamp

and producing:

RAW MATERNAL SENSOR DATA

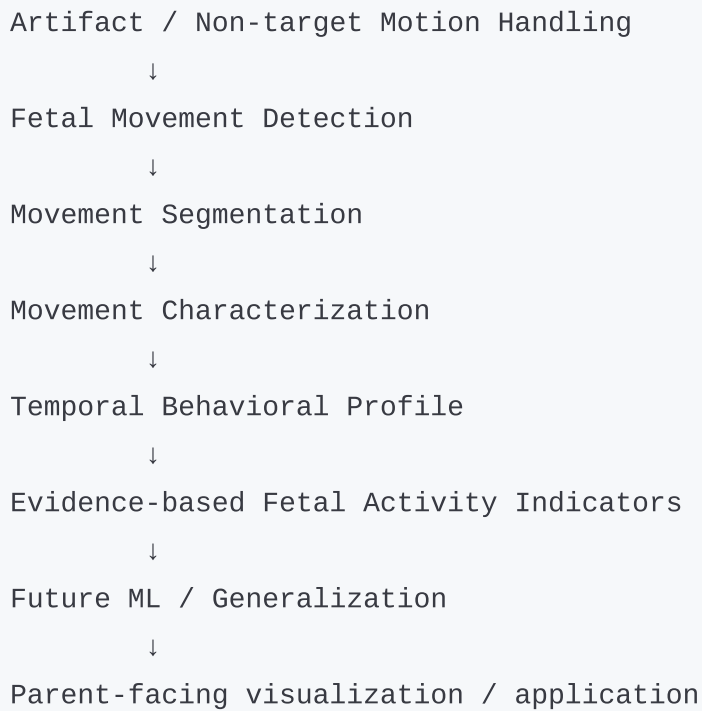


Signal Quality / Timing Validation



Signal Processing





Important scientific boundary

Do **not** currently claim:

"The system diagnoses fetal health."

Instead, the research target should be:

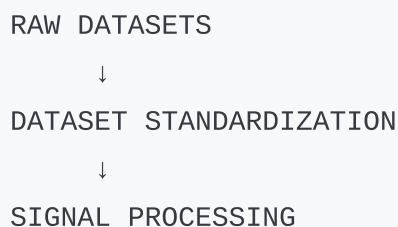
Extracting and characterizing fetal movement behavior and deriving quantitative indicators that may be associated with fetal well-being.

Actual clinical interpretation requires clinical validation.

That distinction will protect the scientific credibility of the project.

1. What We Have Today

Your current architecture is already a good foundation:



↓
ARTIFACT / MOTION QUALITY
↓
MOVEMENT DETECTION
↓
MOVEMENT CHARACTERIZATION
↓
BEHAVIORAL PROFILING
↓
ML / GENERALIZATION

Current datasets:

Four-IMU

Main proxy for:

fetal/maternal-body movement sensing structure.

Provides:

- accelerometer
- gyroscope
- multiple sensor positions
- labels
- subjects/records

Oxford

Provides:

- 3-axis BCG
- fetal-related physiological movement context

It is **not an IMU dataset**, so it should not be forced into the same sensor interpretation.

Cough

Provides:

- accelerometer
- gyroscope
- magnetometer
- timestamps
- trial/subject information
- known activity structure

Its role should be:

supplementary non-target-motion/artifact dataset

not a replacement for actual fetal artifact data.

HRA

Keep it for:

future external/independent validation

Do not force it into the current pipeline merely because it exists.

2. The Biggest Change in Thinking

Previously, the project was approximately:

```
graph TD; Dataset --> Loader; Loader --> Timing; Timing --> Artifact;
```

Now the research system needs to become:

DATA

↓

Can I trust this signal?

↓

Can I process it correctly?

↓

Can I distinguish target movement from contamination?

↓

Can I identify movement events?

↓

Can I describe each event?

↓

Can I describe behavior over time?

↓

Can I derive meaningful indicators?

Every layer must answer **one scientific question**.

PHASE 1 — DATASET STANDARDIZATION

Existing status

Already done

You have:

```
dataset.py
```

with:

```
CoughLoader  
FourIMULoader  
OxfordLoader
```

and a common representation.

This is good.

What must change

Freeze the loader layer.

Do **not** keep modifying the loader every time downstream processing encounters a problem.

The loader should only answer:

"Can I convert this heterogeneous dataset into a predictable representation without destroying information?"

It should NOT:

- filter signals
- detect artifacts
- detect movement
- normalize signals
- create movement features

Required frozen responsibilities

```
RAW FILES
  ↓
Loader
  ↓
standardized dataframe
  +
metadata
```

Each loader must preserve:

- timestamp
- sensor identity
- subject identity
- record identity
- dataset source
- available modalities
- original labels

- sub-dataset information

And:

Never invent unavailable sensor channels.

For Oxford:

```
bcg_x  
bcg_y  
bcg_z
```

remain valid.

IMU fields remain unavailable.

PHASE 2 — DATA EXPLORATION

This phase is where you understand:

What exactly does each dataset contain?

Your notebooks already started this.

But now make the output systematic.

For every dataset document:

Sampling

- nominal sampling rate
- actual sampling interval
- timing irregularity
- missing timestamps
- duplicate timestamps

Sensors

- available axes

- number of sensors
- sensor placement if known
- units

Labels

Most importantly:

What does the original label actually mean?

Do not simply rename a dataset's label to `"fetal_movement"`.

Create a semantic mapping:

```
Original label
    ↓
What it actually represents
    ↓
How useful it is for our pipeline
```

This is particularly important for Cough.

PHASE 3 — TIMING / TEMPORAL INTEGRITY

This needs significant strengthening.

Your timing layer should not just say:

"timestamps exist."

It should establish whether the signal can safely be processed as a time series.

Exact questions

For every recording:

1. Timestamp validity

Are timestamps monotonic?

2. Sampling interval

Calculate:

$$\Delta t = t[i+1] - t[i]$$

3. Sampling frequency

Estimate:

$$fs = 1 / \text{median}(\Delta t)$$

4. Jitter

Measure variation in Δt .

5. Gaps

Identify:

$$\Delta t \gg \text{expected } \Delta t$$

6. Duplicates

Detect:

$$\Delta t = 0$$

7. Continuity

Determine whether a recording contains:

continuous segment

or:

multiple discontinuous segments

PHASE 4 — CONFIGURATION

This is one of the areas you correctly identified as weak.

Create one central configuration system.

For example:

```
config/  
  datasets.yaml  
  signal_processing.yaml  
  artifact.yaml
```

The configuration should contain things such as:

```
sampling rate  
resampling rate  
filter parameters  
window duration  
window overlap  
quality thresholds  
artifact thresholds  
feature parameters
```

The important principle is:

Methods should not be hidden as random constants inside scripts.

Instead:

```
CONFIG  
  ↓  
PROCESSING
```

This makes experiments:

- reproducible
- explainable
- comparable
- easier to modify when real data arrives.

PHASE 5 — SIGNAL PROCESSING

This is currently one of your weakest methodological layers.

We need to make this scientifically defensible before touching the real maternal dataset.

Final signal-processing chain

```
Raw signal
  ↓
Timestamp validation
  ↓
Resampling / common time base
  ↓
Sensor synchronization
  ↓
Unit validation
  ↓
Filtering
  ↓
Signal quality assessment
  ↓
Windowing
  ↓
Analysis-ready windows
```

Why each step exists

Resampling

Different recordings may have slightly different sampling intervals.

Create a common time base.

Synchronization

For multiple sensors:

```
IMU1
IMU2
```

IMU3

IMU4

their signals must correspond to the same time coordinate.

Filtering

The filter must have a **physiological/measurement justification**.

Don't simply say:

"We applied Butterworth filtering."

Document:

- filter type
 - order
 - cutoff
 - why those cutoffs were chosen
 - whether zero-phase filtering is used
 - effect on signal
-

Signal quality

Before declaring:

"movement"

ask:

"Is this segment even trustworthy?"

Possible checks:

- missingness
- saturation

- clipping
 - excessive noise
 - abnormal amplitude
 - timing gaps
 - sensor dropout
-

Windowing

Convert continuous signals into analysis units:

```
continuous signal
    ↓
window 1
window 2
window 3
...
```

The window size and overlap must be configurable.

PHASE 6 — ARTIFACT / NON-TARGET MOTION

This is where your project becomes substantially stronger.

The key question is:

What isn't fetal movement?

Potential contamination:

```
maternal movement
sensor displacement
belt movement
body repositioning
large external motion
sensor noise
recording disturbances
```

Important distinction

Do NOT make this:

```
artifact = no movement
```

Instead:

```
SIGNAL QUALITY
+
NON-TARGET MOTION
+
TARGET MOVEMENT
```

are conceptually different.

Artifact pipeline

```
Processed signal
↓
Candidate event generation
↓
Event representation
↓
Candidate labels
↓
Artifact baseline
↓
Subject-independent evaluation
↓
Artifact detector
```

Why candidate generation previously failed

This is one of the things you need to fix.

Your pipeline must not assume that:

```
candidate_events.csv
```

will magically appear.

You need an explicit deterministic candidate-generation algorithm.

For example conceptually:

```
signal
↓
magnitude / derived representation
↓
activity score
↓
threshold
↓
contiguous active regions
↓
merge close regions
↓
minimum duration
↓
candidate event
```

The exact algorithm should be experimentally justified.

PHASE 7 — ARTIFACT FEATURES

For every candidate event, extract features such as:

Time-domain

- duration
- RMS
- peak amplitude
- peak-to-peak
- variance
- standard deviation
- zero crossing characteristics

Energy

- signal energy
- windowed energy

Frequency-domain

- dominant frequency
- spectral centroid
- band power
- spectral entropy

Multi-axis

- vector magnitude
- axis contribution
- directional consistency

Multi-sensor

- number of sensors involved
- synchronization of sensor responses
- spatial consistency

This converts:

raw waveform

into:

event feature vector

which is what ML can actually consume.

PHASE 8 — ARTIFACT BASELINE

Now `04_artifact_baseline_models.py` becomes meaningful.

The question becomes:

Can simple models distinguish artifact/non-target motion candidates based on the extracted features?

Start simple.

For example:

Logistic Regression
Random Forest
XGBoost

depending on what your available labels support.

But don't chase accuracy first.

Evaluate:

- precision
- recall
- F1
- ROC-AUC where appropriate
- confusion matrix

And most importantly:

subject-independent validation

Do not randomly split windows from the same subject into train/test.

That can cause leakage.

Prefer:

Leave-One-Subject-Out

or subject-grouped cross-validation.

PHASE 9 — MOVEMENT DETECTION

Only after artifact handling.

The question changes from:

"Is this artifact?"

to:

"Is this a genuine fetal movement event?"

This requires proper event segmentation.

Output

For every movement:

```
movement_id
subject_id
record_id
start_time
end_time
duration
```

Then characterization happens downstream.

PHASE 10 — MOVEMENT CHARACTERIZATION

This is where you stop merely **counting kicks**.

For each movement:

Temporal

- onset
- offset
- duration
- inter-event interval

Amplitude

- peak acceleration

- RMS
- energy

Frequency

- dominant frequency
- spectral characteristics

Direction

- x/y/z contribution
- resultant direction
- orientation consistency

Spatial

With multiple IMUs:

Which sensors responded?

This can potentially provide information about movement propagation/spatial patterns.

PHASE 11 — BEHAVIORAL PROFILING

This is your **big research upgrade**.

Instead of:

"There were 18 movements."

you want:

"What pattern of activity did the fetus exhibit during this observation period?"

Build a behavioral profile containing things like:

movement frequency
activity density
inter-event intervals
burstiness
movement duration distribution
movement amplitude distribution
temporal activity distribution
movement repertoire

Potential profile:

LOW ACTIVITY
MODERATE ACTIVITY
HIGH ACTIVITY
BURST-LIKE ACTIVITY
SPARSE ACTIVITY

But these categories should eventually be derived from data rather than arbitrarily imposed.

PHASE 12 — HEALTH INTERPRETATION

This is where you need the most scientific caution.

Your software should initially produce:

Quantitative indicators

rather than:

"Baby is healthy."

For example:

movement rate
activity pattern
movement duration
movement intensity

temporal distribution
movement variability

Then eventually research whether these indicators correlate with clinically meaningful outcomes.

The eventual hierarchy should be:

Movement measurements
↓
Behavioral profile
↓
Validated clinical associations
↓
Potential well-being indicators

Not:

movement
↓
AI
↓
health diagnosis

PHASE 13 — HRA / EXTERNAL VALIDATION

Keep HRA in reserve.

Once the pipeline works:

Four-IMU
Oxford
Cough
↓
development / validation

then:

HRA
↓

external validation

This becomes much more powerful scientifically.

You can say:

"The pipeline was developed and internally validated using multiple datasets and subsequently evaluated on an independent dataset."

Only if the actual experiments support that statement.

PHASE 14 — REAL MATERNAL DATA

This is the immediate deadline.

When your teammate provides:

```
timestamp
ax
ay
az
gx
gy
gz
```

you should ideally only need a new adapter/loader.

The rest should remain unchanged.

This is **critical architecture**.

You want:

```
MPU6050 CSV
  ↓
MaternalLoader
  ↓
COMMON SCHEMA
  ↓
existing pipeline
```

not:

MPU6050 CSV



rewrite entire project

PHASE 15 — HARDWARE ↔ SOFTWARE INTEGRATION

Your teammate already has:

MPU6050



ACC



GYRO



CSV



Flask visualization

Your responsibility is to create:

Hardware output



validated data interface



dataset loader



signal processing



artifact handling



movement analysis

The hardware/software interface should be frozen early.

Define a minimum contract:

timestamp

ax

ay

```
az
gx
gy
gz
```

plus metadata where available.

PHASE 16 — REAL-TIME VS OFFLINE

Don't try to build the complete real-time system immediately.

First make:

Offline pipeline

```
CSV
↓
complete analysis
↓
results
```

Then:

Near-real-time

```
stream/buffer
↓
processing window
↓
prediction
↓
event
```

Then:

Real-time application

```
sensor
↓
edge/device
↓
```

```
backend
  ↓
analysis
  ↓
database
  ↓
mobile application
```

This prevents you from debugging hardware, networking, signal processing and ML simultaneously.

PHASE 17 — FUTURE APPLICATION

Only after the analytical pipeline is reliable.

The future app could present:

For researchers/clinicians

```
Movement count
Movement intensity
Movement duration
Activity timeline
Behavioral profile
Signal quality
Artifact percentage
```

For parents

Avoid overwhelming them with raw biomedical information.

Instead:

```
Fetal activity visualization
  ↓
animated fetal representation
  ↓
movement timeline
  ↓
simple activity summary
```

The "virtual fetus" should be a **visualization of detected movement patterns**, not a literal reconstruction of fetal biomechanics unless you eventually have the data/model to justify that.

Your 7-8 Day Immediate Strategy

This is the part I would personally follow if I were executing the project with your deadline.

Day 1 — Freeze foundation

Finish:

```
dataset.py  
loader_master_test.py
```

Then freeze.

Also document:

```
common schema  
metadata schema  
dataset semantics
```

Do not touch loaders afterward unless a real bug is discovered.

Day 2 — Fix timing + configuration

Complete:

```
01_timing.py  
timing_real_data_test.py
```

Make timing output explicit:

```
sampling rate  
median  $\Delta t$   
jitter  
gaps  
duplicates  
continuity
```

Create central configuration.

Day 3 — Build trustworthy signal processing

Implement and validate:

```
resampling  
synchronization  
filtering  
quality checks  
windowing
```

Generate representative plots.

This day is extremely important.

You need to be able to explain:

"We do this because..."

for every processing step.

Day 4 — Finish artifact candidate generation

This is a major milestone.

You need working:

```
candidate_events.csv  
segment_features.csv  
inspection_summary.csv  
manual_label_template.csv
```

for at least the relevant datasets.

And representative plots.

Day 5 — Artifact baseline

Implement:

```
feature matrix  
labels
```

```
baseline models
subject-independent validation
metrics
```

Don't optimize endlessly.

Get a **reproducible baseline** first.

Day 6 — Movement detection prototype

Build:

```
processed signal
↓
artifact-aware signal/events
↓
movement candidate
↓
movement event
```

Generate:

```
movement_events.csv
```

with:

```
start
end
duration
amplitude
energy
```

etc.

Day 7 — Movement characterization + behavioral profile

Build:

```
movement events
↓
event features
```

```
↓  
session-level statistics  
↓  
behavioral profile
```

Output something like:

```
behavior_profile.json
```

or:

```
behavior_profile.csv
```

Day 8 — Real-data readiness

Take the expected MPU6050 format:

```
timestamp  
ax  
ay  
az  
gx  
gy  
gz
```

and test the entire pipeline with a synthetic/sample file that follows exactly that schema.

The objective:

When the real maternal recording arrives, adding the data should not require redesigning the pipeline.

The Definition of "Done"

Do not define success as:

"I wrote all the Python files."

Define it as:

Input

```
MPU6050 maternal recording
```

Processing

```
loader
↓
timing
↓
signal processing
↓
quality
↓
artifact handling
↓
movement detection
↓
characterization
↓
behavioral profiling
```

Output

```
clean/usable signal
artifact events
movement events
movement features
behavior profile
visualizations
```

And every stage should have:

```
input
↓
method
```

↓
output
↓
validation

The Most Important Rule for the Whole Project

From now on, whenever you add a method, ask these **five questions**:

- 1. What problem does this solve?**
- 2. Why is this method appropriate?**
- 3. What exactly happens to the signal?**
- 4. What comes out of this stage?**
- 5. How do we know that it worked?**

If you cannot answer those five questions, **don't add the method yet.**

Your Final Mental Model

Keep this story in your head:

We first learned what our heterogeneous datasets actually contain.

↓

We built dataset loaders so different datasets can enter the system through one common interface without inventing missing information.

↓

We verified timing because signal processing is meaningless if the temporal structure is unreliable.

↓

We convert raw recordings into analysis-ready signals through synchronization, resampling, filtering, quality assessment and windowing.



We then deal with artifacts because not every movement in the sensor is fetal movement.



Once contaminated/non-target events are controlled, we detect actual movement events.



We don't stop at counting them—we measure their duration, amplitude, energy, frequency, direction and sensor contribution.



Those individual movements are then aggregated over time to form a behavioral profile.



Only after obtaining reliable movement information do we apply ML to learn patterns and evaluate generalization.



Eventually, validated movement behavior can become quantitative indicators of fetal well-being and be presented through an intuitive application.

And the immediate engineering principle is:

Build the pipeline today so that tomorrow's real maternal MPU6050 data becomes just another input—not a reason to redesign the system.

That should be your **north-star strategy** for the next 7-8 days.

8/21/2026, 10:01:41 PM